

Technologie Blockchain

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction : principes fondamentaux de la blockchain

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définition

Une blockchain est un registre distribué, répliqué sur de nombreux nœuds d'un réseau pair-à-pair, structuré en blocs chaînés cryptographiquement, dont le contenu passé est rendu extrêmement difficile à modifier une fois validé, sans nécessiter d'autorité centrale unique de confiance.

2. Genèse : Bitcoin (2008-2009)

Publié sous le pseudonyme Satoshi Nakamoto en 2008, Bitcoin est la première application combinant, de façon inédite, des briques cryptographiques déjà connues (signatures numériques, fonctions de hachage) avec un mécanisme de consensus décentralisé (preuve de travail, chapitre 3) pour résoudre le **problème de la double dépense** sans tiers de confiance central : comment empêcher qu'une même unité de monnaie numérique soit dépensée deux fois, sans banque centrale ?

3. Propriétés recherchées

Propriété	Signification
Décentralisation	aucune entité unique ne contrôle le registre ; il est répliqué et validé par de nombreux participants indépendants
Immutabilité (pratique)	modifier un bloc déjà validé et suffisamment confirmé nécessiterait de refaire le travail de validation de tous les blocs suivants, un coût prohibitif au-delà d'une certaine profondeur
Transparence	(pour les blockchains publiques) l'historique complet des transactions est consultable par tous
Résistance à la censure	aucune autorité unique ne peut empêcher une transaction valide d'être incluse, dans un réseau suffisamment décentralisé

Ces propriétés sont des objectifs de conception, pas des garanties absolues : elles dépendent fortement du degré réel de décentralisation du réseau (chapitre 3) et de la sécurité des implémentations (chapitre 5).

4. Blockchain publique, privée, à permission

Type	Accès en écriture	Accès en lecture	Exemple
Publique (permissionless)	ouvert à tous	ouvert à tous	Bitcoin, Ethereum
À permission (consortium)	limité à des participants identifiés	selon règles définies	réseaux inter-bancaires, chaînes logistiques inter-entreprises
Privée	contrôlé par une seule organisation	souvent restreint	usage interne d'entreprise, proche d'une base de données répliquée classique

Une confusion fréquente : une blockchain « privée » ou « à permission » perd une grande partie des propriétés de décentralisation et de résistance à la censure d'une blockchain publique — le choix du type doit être justifié par le besoin réel, pas par un effet de mode.

5. Anatomie d'un bloc (aperçu)

Chaque bloc contient typiquement : un en-tête (empreinte du bloc précédent, horodatage, racine de Merkle des transactions du bloc — chapitre 2, un nonce pour le mécanisme de consensus), et le corps (liste des transactions incluses).

6. Ce que la blockchain résout, et ce qu'elle ne résout pas

- Elle résout la **cohérence d'un registre partagé sans tiers de confiance unique**, dans un contexte où les participants peuvent être mutuellement méfiants.
- Elle ne résout pas, par elle-même, la **véracité des données saisies** (« garbage in, garbage out ») : une blockchain garantit l'intégrité de ce qui y a été inscrit, pas la véracité de l'information au moment de sa saisie (problème dit de l'« oracle », chapitre 4).
- Elle a un coût réel : stockage répliqué, consommation énergétique pour certains mécanismes de consensus (chapitre 3), latence de confirmation, complexité opérationnelle — d'où l'importance d'évaluer si une base de données traditionnelle ne répondrait pas mieux à un besoin donné (chapitre 6).

À retenir

- La blockchain combine des primitives cryptographiques connues avec un mécanisme de consensus décentralisé pour résoudre le problème de la double dépense sans autorité centrale.
- Les propriétés de décentralisation, immutabilité et résistance à la censure sont des objectifs de conception dont le degré réel dépend de l'architecture choisie.
- Une blockchain n'est pas une solution universelle : son usage doit être justifié par un besoin réel de décentralisation, pas adopté par principe.

Chapitre 2 — Structures de données : chaînage cryptographique et arbres de Merkle

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Le chaînage cryptographique des blocs

Chaque bloc contient, dans son en-tête, l'**empreinte de hachage du bloc précédent**. Cette construction crée une dépendance en cascade : modifier le contenu d'un bloc ancien change son empreinte, ce qui invalide l'empreinte enregistrée dans le bloc suivant, et ainsi de suite jusqu'au sommet de la chaîne. Un attaquant souhaitant altérer un bloc ancien devrait donc recalculer (et, selon le mécanisme de consensus, refaire valider) tous les blocs suivants plus rapidement que le reste du réseau honnête — d'où le terme d'immutabilité « pratique » plutôt qu'absolue, et la recommandation d'attendre un nombre suffisant de confirmations avant de considérer une transaction comme définitive.

2. Arbre de Merkle

Un arbre de Merkle organise un ensemble de données (les transactions d'un bloc) de façon à pouvoir résumer l'ensemble par une seule empreinte (la **racine de Merkle**) tout en permettant de prouver efficacement qu'une donnée particulière fait bien partie de l'ensemble, sans avoir à transmettre l'ensemble complet.

Construction

1. Chaque transaction est hachée individuellement (feuilles de l'arbre).
2. Les empreintes sont regroupées deux par deux et hachées ensemble pour former le niveau supérieur.
3. Ce processus est répété jusqu'à obtenir une seule empreinte : la racine de Merkle, stockée dans l'en-tête du bloc.

Preuve de Merkle (Merkle proof)

Pour prouver qu'une transaction **T** appartient à un bloc, il suffit de fournir **T** et la liste des empreintes « sœurs » sur le chemin vers la racine (de l'ordre de $\log_2(n)$ empreintes pour n transactions), plutôt que l'ensemble des n transactions. Cette propriété est essentielle pour les **clients légers** (**SPV — Simplified Payment Verification**), qui vérifient qu'une transaction est incluse dans un bloc sans télécharger l'intégralité de la blockchain.

3. Pourquoi cette double structure (chaîne de blocs + arbre par bloc) ?

- Le **chaînage entre blocs** garantit l'intégrité de l'ordre et de l'historique global.
- L'**arbre de Merkle au sein d'un bloc** garantit l'intégrité de l'ensemble des transactions d'un bloc tout en permettant des vérifications partielles efficaces.

Cette architecture illustre directement l'usage des fonctions de hachage (module *Cryptographie*, chapitre 3) comme brique de construction pour des propriétés de plus haut niveau (intégrité globale d'un registre distribué).

4. Identifiants et adresses

- Une **transaction** est identifiée par l'empreinte de son propre contenu (le « hash de transaction »).
- Une **adresse** (compte) dérive généralement d'une clé publique (elle-même souvent hachée pour former une adresse plus courte), reliant directement ce chapitre au module *Cryptographie* (signatures numériques, chapitre 5) : chaque transaction est signée par la clé privée correspondant à l'adresse d'origine, et cette signature est vérifiable par tout nœud du réseau.

5. Variantes et structures alternatives

- **DAG (Directed Acyclic Graph)** : certaines architectures (ex. IOTA) remplacent la chaîne linéaire de blocs par un graphe orienté acyclique de transactions, visant une meilleure scalabilité, au prix d'autres compromis de sécurité et de maturité.
- **État global (state trie)** : les blockchains à smart contracts (Ethereum, chapitre 4) maintiennent, en plus de l'historique des transactions, une structure arborescente représentant l'état courant de tous les comptes et contrats, également authentifiée cryptographiquement (racine d'état incluse dans chaque bloc).

À retenir

- Le chaînage cryptographique entre blocs et l'arbre de Merkle au sein d'un bloc sont deux mécanismes complémentaires garantissant respectivement l'intégrité de l'historique global et celle du contenu de chaque bloc.
- Une preuve de Merkle permet de vérifier l'inclusion d'une transaction sans télécharger l'intégralité de la blockchain, fondement des clients légers.
- Toute cette architecture repose directement sur les fonctions de hachage et les signatures numériques étudiées dans le module *Cryptographie*.

Chapitre 3 — Mécanismes de consensus

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi un mécanisme de consensus ?

Dans un réseau décentralisé sans autorité centrale, il faut un mécanisme permettant à des participants potentiellement nombreux, anonymes et parfois malveillants de s'accorder sur un état unique et cohérent du registre (quel bloc suit quel bloc), tout en résistant à des tentatives de manipulation. C'est une instance du problème classique dit **des généraux byzantins** en informatique distribuée.

2. Preuve de travail (Proof of Work, PoW)

Principe : pour proposer un nouveau bloc, un participant (« mineur ») doit résoudre un défi calculatoire coûteux — trouver une valeur (le *nonce*) telle que l'empreinte de hachage du bloc satisfasse une condition de difficulté (par exemple, commencer par un certain nombre de zéros). Ce calcul est intentionnellement coûteux à réaliser mais trivial à vérifier par les autres nœuds.

- **Sécurité** : falsifier l'historique nécessiterait de disposer d'une puissance de calcul supérieure à celle du reste du réseau honnête combiné (« attaque des 51 % »), un coût économique dissuasif pour un réseau suffisamment décentralisé et puissant.
- **Limite majeure** : consommation énergétique très importante (Bitcoin), motivant la recherche d'alternatives.

3. Preuve d'enjeu (Proof of Stake, PoS)

Principe : le droit de proposer et de valider un bloc est attribué (souvent par tirage pondéré) en fonction de la quantité de cryptomonnaie qu'un participant a « mise en jeu » (staked), plutôt qu'en fonction de sa puissance de calcul.

- **Sécurité** : un participant malveillant risque de perdre (« slashing ») une partie de sa mise s'il tente de valider des blocs contradictoires ou invalides, ce qui aligne son intérêt économique avec le comportement honnête.
- **Avantage** : consommation énergétique très inférieure à PoW (Ethereum est passé de PoW à PoS en 2022, réduisant sa consommation énergétique de plus de 99 %).
- **Débat en cours** : risque de centralisation accrue au profit des plus gros détenteurs (« les riches deviennent validateurs, les validateurs gagnent des récompenses, les riches s'enrichissent davantage »).

4. Tolérance byzantine pratique (PBFT et variantes)

Utilisée typiquement dans les blockchains à permission (nombre de participants limité et identifié) : les nœuds valident un bloc par un mécanisme de vote explicite, tolérant qu'une fraction des participants (typiquement jusqu'à un tiers) soit défaillante ou malveillante, avec une finalité de validation quasi immédiate (contrairement à PoW où la certitude augmente progressivement avec les confirmations).

5. Comparaison synthétique

Mécanisme	Coût de sécurité	Consommation énergétique	Finalité	Exemple
PoW	puissance de calcul	très élevée	probabiliste (croît avec les confirmations)	Bitcoin
PoS	capital économique en jeu	faible	souvent quasi immédiate selon l'implémentation	Ethereum (depuis 2022)
BFT	identité/réputation des validateurs	faible	immédiate	réseaux à permission, certaines blockchains d'entreprise

6. Fork et résolution de conflits

Lorsque deux blocs valides sont proposés presque simultanément (situation normale dans un réseau distribué), une **fourche (fork)** temporaire apparaît ; les nœuds convergent généralement vers la chaîne la plus longue (PoW) ou celle validée par le plus de participants (PoS) selon la règle de choix définie par le protocole. Un **hard fork** délibéré (changement de règles incompatible) peut aussi survenir lors d'une mise à jour majeure ou d'un désaccord de gouvernance dans la communauté.

À retenir

- Le mécanisme de consensus répond à un problème classique d'informatique distribuée (accord dans un réseau potentiellement malveillant, sans autorité centrale).
- PoW sécurise par le coût calculatoire mais consomme beaucoup d'énergie ; PoS sécurise par un risque économique direct (slashing) avec une empreinte énergétique bien moindre.
- Le choix du mécanisme de consensus détermine directement les propriétés de sécurité, de décentralisation réelle et de coût du réseau — un compromis, pas une hiérarchie absolue de qualité.

Chapitre 4 — Smart contracts et Ethereum

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Qu'est-ce qu'un smart contract ?

Un smart contract (« contrat intelligent ») est un programme informatique déployé sur une blockchain, dont le code et l'état s'exécutent de façon déterministe et vérifiable par l'ensemble des nœuds du réseau, sans possibilité de modification unilatérale une fois déployé (sauf mécanisme d'évolutivité explicitement prévu). Le terme « contrat » est trompeur au sens juridique : il s'agit avant tout d'un **programme auto-exécutable**, dont la portée juridique effective dépend du contexte réglementaire.

2. Ethereum : une blockchain programmable

Ethereum (2015) a étendu le modèle de Bitcoin en introduisant une **machine virtuelle Turing-complète** (l'EVM — Ethereum Virtual Machine), permettant d'exécuter des programmes arbitrairement complexes sur la blockchain, et non plus seulement des transactions de transfert de valeur.

Notion de « gas »

Chaque opération exécutée par l'EVM consomme du « gas », payé par l'émetteur de la transaction dans la cryptomonnaie native (Ether). Ce mécanisme a un double rôle : rémunérer les validateurs pour les ressources de calcul consommées, et **empêcher les boucles infinies ou les attaques par déni de service** (un programme épuise son budget de gas avant de pouvoir consommer des ressources indéfiniment — solution pratique au problème de l'arrêt appliqué à un contexte adversarial).

3. Langages et environnement de développement

- **Solidity** : langage le plus utilisé pour écrire des smart contracts sur Ethereum, syntaxe proche du JavaScript/C++.
- **Environnements de développement/test** : Remix (IDE en ligne), Hardhat, Foundry, permettant de développer, tester et déployer sur des réseaux de test avant tout déploiement sur le réseau principal.
- **Réseaux de test (testnets)** : répliques du réseau principal utilisant une monnaie sans valeur réelle, indispensables pour tester un contrat avant déploiement (utilisés en TP3/TP4).

4. Exemple simplifié de structure d'un smart contract (Solidity)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract CompteSimple {
    mapping(address => uint256) public soldes;

    function deposer() external payable {
        soldes[msg.sender] += msg.value;
    }

    function retirer(uint256 montant) external {
        require(soldes[msg.sender] >= montant, "Solde insuffisant");
        soldes[msg.sender] -= montant;
        payable(msg.sender).transfer(montant);
    }
}
```

Ce court exemple illustre déjà des notions clés : état persistant sur la blockchain (`mapping`), vérifications explicites (`require`), et manipulation de valeur (Ether) — et prépare directement le chapitre suivant sur la sécurité (l'ordre des opérations dans `retirer` a une importance de sécurité directe).

5. Cas d'usage des smart contracts

- **Jetons (tokens)** : représentation d'actifs numériques selon des standards (ERC-20 pour des jetons fongibles, ERC-721 pour des jetons non fongibles/NFT).
- **Finance décentralisée (DeFi)** : prêts, échanges décentralisés (DEX), sans intermédiaire financier traditionnel.
- **Gouvernance décentralisée (DAO)** : votes et décisions automatisées selon des règles codées.
- **Traçabilité** : suivi de chaîne logistique, certification de provenance.

6. Le problème de l'oracle

Un smart contract, par conception, ne peut accéder qu'aux données déjà présentes sur la blockchain : il ne peut pas consulter directement une donnée du monde extérieur (cours d'une action, résultat sportif, météo). Un **oracle** est un service tiers chargé d'apporter cette donnée externe sur la blockchain. Ce point est une source de risque majeure : la sécurité de tout le smart contract dépend alors de la fiabilité de l'oracle utilisé, qui redevient un point de confiance centralisé, en tension avec l'objectif initial de décentralisation.

| À retenir

- Un smart contract est un programme auto-exécutable, pas nécessairement un contrat juridique au sens classique.
- Le mécanisme de gas rémunère l'exécution et empêche les boucles infinies, brique essentielle de la sécurité opérationnelle d'Ethereum.
- Le problème de l'oracle rappelle qu'un smart contract ne résout la confiance que pour les données déjà présentes sur la chaîne, pas pour les données du monde réel qui y sont importées.

Chapitre 5 — Sécurité et vulnérabilités des smart contracts

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Pourquoi la sécurité des smart contracts est particulièrement critique

Un smart contract déployé sur une blockchain publique est, par construction, **immuable et son code est public** : contrairement à une application classique, il n'est généralement pas possible de corriger une faille par un simple correctif déployé discrètement — le code vulnérable reste actif et visible de tous jusqu'à une migration complexe. De plus, les contrats manipulent souvent directement des fonds réels, ce qui en fait des cibles à forte valeur immédiate pour un attaquant.

2. Réentrance (reentrancy)

Vulnérabilité historique majeure (à l'origine du piratage « The DAO » en 2016, environ 60 millions de dollars de l'époque). Principe : un contrat appelle un contrat externe (ex. pour transférer des fonds) **avant** d'avoir mis à jour son propre état interne ; le contrat externe malveillant peut alors rappeler la fonction d'origine de façon récursive, exploitant le fait que l'état (le solde) n'a pas encore été mis à jour.

```
// Version vulnérable
function retirer(uint256 montant) external {
    require(soldes[msg.sender] >= montant, "Solde insuffisant");
    payable(msg.sender).transfer(montant); // appel externe AVANT la mise à jour de l'état
    soldes[msg.sender] -= montant;          // trop tard : un contrat malveillant a pu rappeler retirer() entre-temps
}
```

Contre-mesure : appliquer le patron **checks-effects-interactions** (vérifications, puis mise à jour de l'état interne, puis seulement interaction externe) — c'est exactement l'ordre correct déjà utilisé dans l'exemple du chapitre 4.

3. Débordement/souppassement d'entier (overflow/underflow)

Historiquement, un compteur non signé en Solidity pouvait « déborder » silencieusement (revenir à zéro après le maximum, ou à une valeur maximale après une soustraction sous zéro), exploité pour créer des soldes ou des quantités de jetons artificiellement énormes. Depuis Solidity 0.8, ces vérifications sont effectuées automatiquement par défaut (le compilateur annule la transaction en cas de dépassement), mais le risque reste réel pour du code utilisant des versions antérieures ou des blocs `unchecked` mal justifiés.

4. Contrôle d'accès défaillant

Une fonction sensible (ex. modification du propriétaire du contrat, retrait de fonds réservés à l'administrateur) sans vérification appropriée des droits de l'appelant (`require(msg.sender == proprietaire)`) permet à n'importe quel utilisateur de l'exécuter. C'est l'équivalent, dans le monde des smart contracts, du contrôle d'accès défaillant classé numéro un de l'OWASP Top 10 (module *Audit organisation et technique*).

5. Attaques par manipulation d'oracle de prix

Un contrat DeFi qui détermine un prix d'actif à partir d'une source manipulable (ex. la liquidité instantanée d'un seul échange décentralisé) peut être trompé par un attaquant qui manipule temporairement ce prix (souvent via un **prêt flash**, un emprunt sans garantie remboursé dans la même transaction) pour déclencher des conditions favorables artificielles dans le contrat cible.

6. Attaques par déni de service et boucles coûteuses en gas

Une fonction itérant sur une structure de données dont la taille peut être manipulée par un attaquant (ex. une liste d'utilisateurs qu'il peut faire croître) peut consommer plus de gas que le maximum autorisé par bloc, rendant la fonction définitivement inexécutable — un déni de service permanent et non corrigible sans migration du contrat.

7. Démarche d'audit d'un smart contract

1. Revue manuelle du code, en confrontant systématiquement chaque fonction sensible aux classes de vulnérabilités ci-dessus.
2. Analyse statique automatisée (outils dédiés type Slither, MythX).
3. Tests unitaires et de fuzzing (génération automatique d'entrées pour détecter des comportements inattendus).
4. Vérification formelle pour les contrats à très haute valeur (preuve mathématique de propriétés de sécurité).
5. Programme de récompense pour la découverte de vulnérabilités (*bug bounty*) avant et après déploiement.

À retenir

- La réentrance, le contrôle d'accès défaillant et la manipulation d'oracle sont des classes de vulnérabilités récurrentes, à l'origine de pertes financières réelles considérables.
- L'immutabilité des smart contracts déployés rend la prévention (audit avant déploiement) bien plus critique que pour une application web classique corrigible a posteriori.
- Le patron checks-effects-interactions est la contre-mesure de référence contre la réentrance, illustrant l'importance de l'ordre des opérations dans le code.

Chapitre 6 — Applications, enjeux et limites

Technologie Blockchain

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Panorama des applications au-delà des cryptomonnaies

- **Finance décentralisée (DeFi)** : prêts, échanges, produits dérivés sans intermédiaire financier centralisé — avec les risques de sécurité vus au chapitre 5 et une volatilité/régulation encore incertaine.
- **Traçabilité de chaîne logistique** : suivi de provenance de produits (agroalimentaire, textile, minerais), où la blockchain garantit l'intégrité des enregistrements une fois saisis, sans garantir la véracité de la saisie initiale (problème de l'oracle, rappel chapitre 4).
- **Identité numérique décentralisée (self-sovereign identity)** : permettre à un individu de contrôler et prouver sélectivement des attributs de son identité sans dépendre d'un registre central unique.
- **Certification et notarisation** : preuve d'existence et d'horodatage d'un document à une date donnée (ancrage de l'empreinte du document sur une blockchain publique) — usage simple et robuste, sans manipuler de valeur financière.
- **Vote électronique** : cas d'usage souvent évoqué mais particulièrement délicat en pratique, car il combine des exigences difficilement conciliables (anonymat de l'électeur, vérifiabilité publique du résultat, résistance à la coercition) — objet de recherche active plutôt que de déploiements matures à grande échelle.

2. Trilemme de la blockchain (scalabilité, sécurité, décentralisation)

Les concepteurs de blockchains publiques font face à un compromis reconnu : il est difficile d'optimiser simultanément la **scalabilité** (nombre de transactions traitées par seconde), la **sécurité** et la **décentralisation**. Les architectures existantes privilégient généralement deux de ces trois propriétés au détriment de la troisième (ex. Bitcoin privilégie sécurité et décentralisation au prix d'une faible scalabilité ; certaines blockchains à permission privilégient scalabilité et sécurité au prix d'une décentralisation réduite).

Approches de mise à l'échelle (aperçu)

- **Solutions de couche 2 (Layer 2)** : traiter la majorité des transactions hors chaîne principale, en n'ancrant sur la chaîne principale qu'un résumé périodique (ex. rollups sur Ethereum).
- **Sharding** : répartir l'état et le traitement entre plusieurs sous-réseaux (« shards ») traités en parallèle.

3. Enjeux réglementaires

- **Lutte contre le blanchiment (AML) et connaissance du client (KYC)** : tension entre pseudonymat des blockchains publiques et obligations réglementaires imposées aux plateformes d'échange.
- **Qualification juridique des cryptoactifs** : selon les juridictions, un jeton peut être qualifié différemment (monnaie, valeur mobilière, actif numérique sui generis), avec des conséquences fiscales et réglementaires très différentes.
- **Protection des consommateurs** : volatilité, risque de perte totale et irréversible en cas d'erreur de manipulation (perte de clé privée) ou d'escroquerie (absence de recours équivalent à un système bancaire traditionnel).
- **Encadrement progressif** : plusieurs juridictions ont mis en place ou développent des cadres réglementaires dédiés aux actifs numériques et aux prestataires de services associés ; la situation évolue rapidement et diffère fortement d'un pays à l'autre.

4. Limites techniques et environnementales

- Consommation énergétique significative pour les réseaux en preuve de travail (chapitre 3), bien que la transition vers la preuve d'enjeu réduise fortement cet impact pour les réseaux concernés.
- Irréversibilité des transactions : un avantage en termes de finalité, mais un risque important en cas d'erreur humaine ou de vol (pas de « bouton d'annulation »).
- Complexité d'usage pour l'utilisateur final (gestion de clés privées, absence de recours en cas de perte), un frein réel à l'adoption grand public.

5. Regard critique : quand la blockchain est-elle pertinente ?

Une grille de questions utile avant d'adopter une solution blockchain pour un projet donné : a-t-on réellement besoin de décentralisation, ou une base de données classique avec un tiers de confiance suffit-elle ? Plusieurs parties mutuellement méfiantes doivent-elles partager un registre sans intermédiaire ? L'immutabilité est-elle un avantage réel pour ce cas d'usage, ou une contrainte gênante (ex. droit à l'effacement en matière de données personnelles) ? De nombreux projets « blockchain » annoncés ces dernières années auraient pu être résolus plus simplement par une architecture centralisée classique.

| À retenir

- Les applications de la blockchain dépassent les cryptomonnaies mais restent souvent confrontées au problème de l'oracle et au trilemme scalabilité/sécurité/décentralisation.
- Le cadre réglementaire des cryptoactifs est encore en construction et varie fortement selon les juridictions.
- La pertinence d'une solution blockchain doit être évaluée par rapport à un besoin réel de décentralisation, pas adoptée par principe.